

Spikingformer+Spik4lite Demo

Introduction

This is a demo implementation of "**Spik4lite: Refactoring Neuromorphic Sparsity for Efficient Spiking Neural Networks on Commodity Edge Devices**". The demo follows the **train-and-infer** pipeline to show the basic usage of Spikingformer+Spik4lite.

Directory

The **Spikingformer** is the root and its structure is described as below.

```
Spikingformer/
├── energy_consumption_calculation/
├── README.md
├── cifar10
│   ├── cifar10.yml
│   ├── model.py
│   ├── SOPs_consumption_on_cifar10.py
│   ├── Spikingformer Power profile.csv
│   ├── Spikingformer+Spik4lite Power profile.csv
│   ├── test.py
│   ├── train.log
│   └── train.py
├── cifar100
│   ├── cifar100.yml
│   ├── model.py
│   ├── SOPs_consumption_on_cifar100.py
│   ├── test.py
│   └── train.py
├── cifar10-dvs
│   ├── autoaugment.py
│   ├── model.py
│   ├── SOPs_consumption_on_CIFAR10DVS.py
│   ├── test.py
│   ├── train.py
│   └── utils.py
├── dvs128-gesture
│   ├── autoaugment.py
│   ├── model.py
│   ├── SOPs_consumption_on_DVS128Gesture.py
│   ├── test.py
│   ├── train.log
│   ├── train.py
│   └── utils.py
```

- * **README.md** provides the general overview and introduction of the project, as well as instructions on how to run the code.

Running the Demo

The whole pipeline contains three major steps: (1) activate the Conda Environment, (2) Run the train code and Run the SOPs consumption code.

(1) activate the Conda Environment

```
cd Spikingformer
conda activate Spik4lite
```

(2) Run the code

Runing on CIFAR10

Setting hyper-parameters in cifar10.yml

```
cd cifar10  
python train.py
```

Runing on CIFAR100

Setting hyper-parameters in cifar100.yml

```
cd cifar100  
python train.py
```

Runing on DVS128 Gesture

```
cd dvs128-gesture  
python train.py
```

Runing on CIFAR10-DVS

```
cd cifar10-dvs  
python train.py
```

Runing on SOPs consumption

```
python SOPs_consumption_on_cifar10.py
```